COGS 181 - Homework 4

due Thursday, February 19 at 11:59PM

Problem 1. (28 points) Conceptual Questions

1.1 (4 points) For image recognition problems (e.g. detecting a cat in a photo), which architecture of neural network would be best suited to solve the problem?

A. 2-layer neural networks.

B. Convolutional Neural Network (CNN).

C. 4-layer neural networks.

D. Perceptron.

Solution: B. Convolutional Neural Network (CNN).

1.2 (4 points) Which of the following gives non-linearity to a neural network?

A. Stochastic Gradient Descent (SGD).

B. Rectified Linear Unit (ReLU).

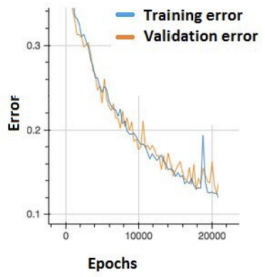
C. Convolutional layer (Conv).

D. Average Pooling layer (AvgPool).

Solution: B. Rectified Linear Unit (ReLU).

1.3 (4 points) In the figure below, we observe that the error curves (both training and validation) have many "ups and downs":

Training and Validation Error Curves



Should we worry about the training process? Choose the **best** explanation below:

A. Yes, because the "ups and downs" means there is a problem with the learning rate of the neural network.

B. Yes, because the "ups and downs" means there is a problem with the model design of the neural network.

C. No, as long as there is a cumulative decrease in both training and validation errors, we do not need to worry.

D. No, since the error is too low (< 0.2 after 10,000 epochs), we do not need to worry about the oscillation of the error.

Solution: C. No, as long as there is a cumulative decrease in both training and validation errors, we do not need to worry.

1.4 (4 points) Which of the following is **the most probable** about model capacity (model capacity means the ability of the neural network to approximate complex functions)?

A. As the number of data points in a mini-batch increases, model capacity increases.

B. As the training loss after convergence increases, model capacity increases.

C. As the learning rate increases, model capacity increases.

D. As the number of learnable parameters increases, model capacity increases.

Solution: D. As the number of learnable parameters increases, model capacity increases.

1.5 (4 points) There is a dataset of 28 × 28 pixel handwritten digits nicely prepared to train a neural network, but we accidentally scrambled the images by re-ordering the pixels in some totally meaningless way, and now we can't get the original dataset back! Below is an image of a handwritten '2' before and after being scrambled.



Figure 1: An image of a handwritten '2' before and after being scrambled.

Luckily, all of the images (in both the training set and the test set) were changed **in the same way**. For example, if pixels number 1 and number 3 switched places in one image, then they switched places in every other image as well. Because of that, a student thinks that a neural network can still be successfully trained after all. Whether that student is right or not depends largely on the type of neural network that the student has in mind. Which of the following neural networks will be at a disadvantage because of the mistake? Choose **all** that apply.

A. A feed-forward neural network with no hidden layer and no convolution layer.

B. A convolutional neural network where the size of each weight filter is 8 × 8 (stride of 1 and no padding).

C. A convolutional neural network where the size of each weight filter is 28 × 28 (stride of 1 and no padding).

D. None of the above.

Solution: B and C

1.6 (4 points) A Convolutional Neural Network (CNN) has 3 consecutive 5×5 convolutional layers with a stride of 1 without pooling. What is the size of the receptive field for a neuron (the set of input image pixels that activate the neuron) in the 3rd convolutional layer?

A. 5.

B. 9.

C. 13.

D. 125.

Solution: C. 13

1.7 (4 points) Choose one that is **least likely** to help reduce overfitting in neural network:

A. Dropout

B. Batch Normalization

C. Momentum

D. L2 Regularization

Solution: C. Momentum

Problem 2. (15 points) Activation Functions

For each activation on the left-hand side, find and connect the corresponding plot of the activation function on the right-hand side.

(a) ReLU

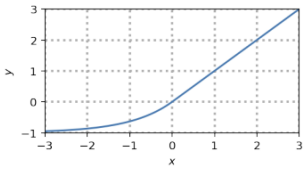
(b) LeakyReLU

(c) Sigmoid

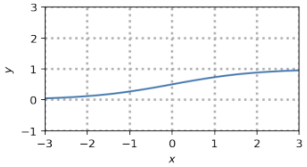
(d) Tanh

(e) ELU

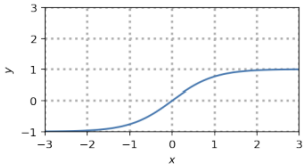
(1)



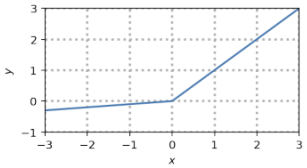
(2)



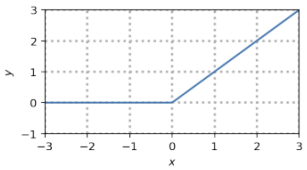
(3)



(4)



(5)



Solution: a - 5, b - 4, c - 2, d - 3, e - 1

Problem 3. (18 points) Convolution neural networks

The 1D convolution operation (we compute the **correlation** here actually) is defined as:

$$h(i) = f(i) * g(i) = \sum_{j=-\infty}^{\infty} f(i+j) \times g(j), \tag{1}$$

where f is a 1-dimensional input vector and g is a convolution filter (kernel). $h(i)$ is often called a response map. The Rectified Linear Unit operation on variable a is defined as:

$$ReLU(a) = \max(0, a). \tag{2}$$

The Max Pooling (window size=2, stride=1) operation on an output map $q(i)$ is defined as:

$$Max(q, i) = \max(q(i), q(i+1)). \tag{3}$$

The Average Pooling (window size=2, stride=1) operation on an output map $q(i)$ is defined as:

$$Avg(q, i) = \frac{q(i) + q(i+1)}{2}. \tag{4}$$

Given an input data vector f:

i	1	2	3	4	5
$f(i)$	10	50	30	20	10

The convolution filter (kernel) g is defined as:

i	-1	0	1
$g(i)$	-2	0	1

Please fill the table below, by sequentially implementing operations including: convolution, activation, and pooling. Note that the stride of convolution is 1, $MaxPool(ReLU(h), i)$ and $AvgPool(ReLU(h), i)$ are **two alternative operations** that both take in $ReLU(h(i))$.

i	0	1	2	3	4	5	6
$f(i)$	0	10	50	30	20	10	0
$h(i) = f(i) * g(i)$	gray	50	10	-80	-50	-40	gray
$ReLU(h(i))$	gray	50	10	0	0	0	gray
$MaxPool(ReLU(h), i)$	gray	50	10	0	0	gray	gray
$AvgPool(ReLU(h), i)$	gray	30	5	0	0	gray	gray

Problem 4. (9 points) 1x1 Convolution

1×1 convolution, also called 1-by-1 convolution, is a kind of convolutional layer which has kernel size 1×1. Although the kernel size is only 1×1, it does not mean that this convolution can only scale the input: When the input has multiple channels, the 1×1 convolution will linearly transform the input channels to output channels for each spatial location. Thus, you can observe that after the 1×1 convolution, the height and width of the output are the same as those of the input, but the channels have changed. More details about 1×1 convolution can be found from http://cs231n.github.io/convolutional-networks/#conv.

In this section, assume the input image $x \in \mathbb{R}^{C \times H \times W}$ where C, H, and W stand for channels, height, and width of the image. We use `Conv-1x1`(m, n) to represent the 1×1 convolutional layer with input channels m and output channels n. Essentially, `Conv-1x1`(m, n) refers to n 1-by-1 convolution filters of size $m \times 1 \times 1$.

4.1 (6 points) Output Shape Inference

If the network is formed as below:

$$\text{Image } \mathbf{x} \rightarrow \texttt{Conv-1x1(C, C1)} \xrightarrow{\mathbf{z}_1} \texttt{Conv-1x1(C1, C2)} \rightarrow \mathbf{z}_2$$

Please calculate the shape of outputs and fill in the blanks below:

1. Shape of input $\mathbf{x}$: $C \times H \times W$.
2. Shape of intermediate output $\mathbf{z}_1$: _____ × _____ × _____.
3. Shape of final output $\mathbf{z}_2$: _____ × _____ × _____.

Solution:

1. Shape of input $\mathbf{x}$: $C \times H \times W$.
2. Shape of intermediate output $\mathbf{z}_1 : C_1 \times H \times W$.
3. Shape of final output $\mathbf{z}_2 : C_2 \times H \times W$.

4.2 (3 points) Model Capacity of Network

In this subsection, we define the **model capacity** as the ability of the neural network to approximate complex functions. For example, assume set S_A contains all the functions that can be represented by network A and set S_B contains all the functions that can be represented by network B. If $S_A = S_B$, it means network A and network B have the **same** model capacity. If $S_A \subset S_B$, it means network A has **higher** model capacity than network B.

Note: The definition of model capacity may vary under different contexts. Please stick to the definition above to solve the question below.

If we have two networks:

- Network 1: Image $\mathbf{x} \rightarrow$ `Conv-1x1(C, C)` $\rightarrow$ `Conv-1x1(C, C)` $\rightarrow \mathbf{z}$
- Network 2: Image $\mathbf{x} \rightarrow$ `Conv-1x1(C, C)` $\rightarrow \mathbf{z}$

Does the network 1 have **higher** capacity than network 2, or the **same** model capacity as network 2? Why?

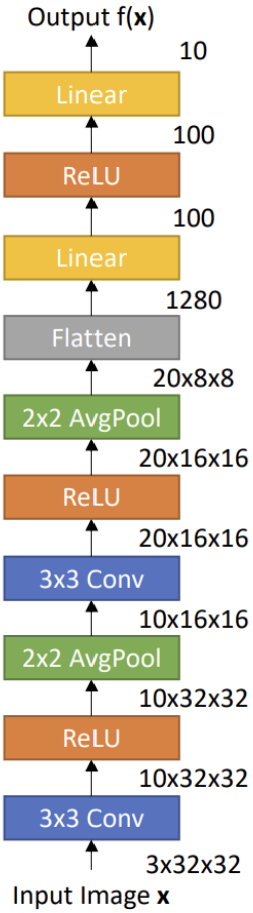
Solution: The network 1 has the **same** capacity as network 2 since the `Conv-1x1(C, C)` can be represented by a single `Conv-1x1(C, C)`.

Problem 5. (30 points) Train a Convolutional Neural Network

In this question, we are going to train a convolutional neural network (CNN) to classify the CIFAR-10 dataset $S = \{(\mathbf{x}^{(i)}, y^{(i)})\}$ where $\mathbf{x}^{(i)} \in \mathbb{R}^{3\times32\times32}$ and $y^{(i)} \in \{0, 1, ..., 9\}$. Assume the images are already normalized. The network structure of the CNN is given on the right, with the shape of the intermediate output marked next to the arrow:

1. For an input image $\mathbf{x}$, we apply two convolutional blocks at first. Each convolutional block has one 3×3 convolutional layer, one ReLU activation layer, and one 2×2 average pooling layer. The two convolutional layers have stride 1 and padding size 1. The intermediate output after two convolutional blocks has the shape $20 \times 8 \times 8$.
2. Then, we flatten the previous output from shape $20 \times 8 \times 8$ into 1280-dimensional vectors. The flattened output will be passed through one linear layer, one ReLU activation layer, and another linear layer.

Besides, we use cross-entropy as the loss function and the stochastic gradient descent (SGD) as the optimizer.



Note:

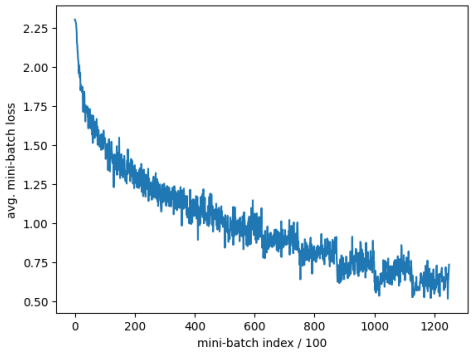
1. For the other details of convolutional layers and linear layers (e.g. in-channels and out-channels), you can infer from the network structure on the right.
2. The shape mentioned above is for a single image. However, in the implementation, you need to deal with a mini-batch of images. Thus, you may find that the shape of tensors in the network has one more dimension (e.g. one input image has shape $3 \times 32 \times 32 \rightarrow$ a mini-batch containing 4 input images has shape $4 \times 3 \times 32 \times 32$).
3. You may notice that there is no softmax layer put on top of the network structure on the right. The reason is that we use `nn.CrossEntropyLoss()` as the loss function, which already has a built-in softmax function.

Download the skeleton code `hw4.ipynb` from the course website. Complete the skeleton code. In order to get full marks, your report should include:

1. Your code.
2. Training loss curve.
3. Test error (overall).
4. Test error (each class).

Hint: You may find the official PyTorch tutorial helpful before implementing the network

Solution: Code is attached at the end.



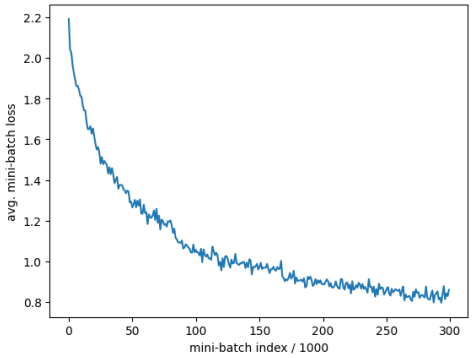
My overall test error is 35%. By class: plane - 25%, car - 27%, bird - 54%, cat - 56%, deer - 30%, dog - 56%, frog - 36%, horse - 29%, ship - 19%, truck - 21%.

Problem 6. (Extra Credit - 15 points) Improve the performance of the CNN on the CIFAR-10 dataset

The problem setting is the same as Problem 5. You can use the same skeleton code `hw4.ipynb`. Try to modify the architecture of the previous one and improve the performance. You can also make changes other than the architecture such as loss function, data preprocessing, etc. Your report should include:

1. Your code.
2. Training loss curve.
3. Test error (overall).
4. Test error (each class).
5. Your main modification. You're encouraged to provide a detailed description and analysis.

Solution: Code is attached at the end.



After a lot of tinkering, I managed to improve my overall test error to 33%. By class: plane - 32%, car - 22%, bird - 49%, cat - 51%, deer - 36%, dog - 46%, frog - 27%, horse - 34%, ship - 22%, truck - 25%.

To improve on the baseline CNN, I first increased model capacity by adding a third conv block and expanding the channel progression to $3 \to 32 \to 64 \to 128$. I inserted BatchNorm2d after each conv layer to stabilize training and speed up convergence. I also replaced average pooling with max pooling to better preserve strong activations. To reduce overfitting, I introduced dropout layers. Together, these changes raised test accuracy from about 65% to 67%, with the largest gains on car, ship, and frog, while cat and bird remained the most difficult classes, as is typical for CIFAR-10.

Problem 5

In [1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
import torch
import torchvision
import torchvision.transforms as transforms
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

Prepare for Dataset

In [2]:
```
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                          shuffle=True, num_workers=2)

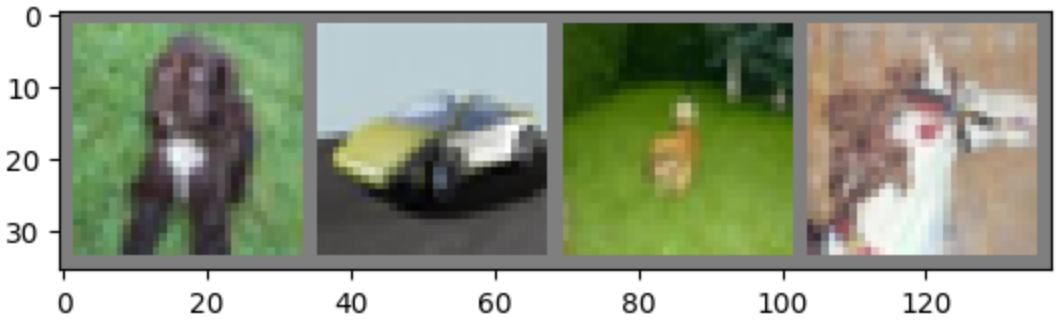
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
                                         shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

```
100%|██████████| 170M/170M [00:16<00:00, 10.5MB/s]
```

In [3]:
```
# The function to show an image.
def imshow(img):
    img = img / 2 + 0.5     # Unnormalize.
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# Get some random training images.
dataiter = iter(trainloader)
images, labels = next(dataiter)
# Show images.
imshow(torchvision.utils.make_grid(images))
# Print labels.
print(' '.join('%5s' % classes[labels[j]] for j in range(4)))
```



```
dog   car   dog horse
```

Choose a Device

In [4]:
```python
# If there are GPUs, choose the first one for computing. Otherwise use CPU.
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(device)
# If 'cuda:0' is printed, it means GPU is available.
```

```
cuda:0
```

Network Definition

In [5]:
```python
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        ###### Fill the blank here ######
        self.conv1 = nn.Conv2d(3, 10, 3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(10, 20, 3, stride=1, padding=1)
        self.fc1 = nn.Linear(1280, 100)
        self.fc2 = nn.Linear(100, 10)

    def forward(self, x):
        ###### Fill the blank here ######
        x = F.relu(self.conv1(x))
        x = F.avg_pool2d(x, 2)
        x = F.relu(self.conv2(x))
        x = F.avg_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

net = Net()     # Create the network instance.
net.to(device)  # Move the network parameters to the specified device.
```

Out[5]:
```
Net(
  (conv1): Conv2d(3, 10, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(10, 20, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (fc1): Linear(in_features=1280, out_features=100, bias=True)
  (fc2): Linear(in_features=100, out_features=10, bias=True)
)
```

Optimizer and Loss Function

In [6]:
```
# We use cross-entropy as loss function.
loss_func = nn.CrossEntropyLoss()
# We use stochastic gradient descent (SGD) as optimizer.
opt = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

Training Procedure

In [7]:
```
avg_losses = []   # Avg. losses.
epochs = 10       # Total epochs.
print_freq = 100  # Print frequency.

for epoch in range(epochs):  # Loop over the dataset multiple times.
    running_loss = 0.0       # Initialize running loss.
    for i, data in enumerate(trainloader, 0):
        # Get the inputs.
        inputs, labels = data

        # Move the inputs to the specified device.
        inputs, labels = inputs.to(device), labels.to(device)

        # Zero the parameter gradients.
        opt.zero_grad()

        # Forward step.
        outputs = net(inputs)
        loss = loss_func(outputs, labels)

        # Backward step.
        loss.backward()

        # Optimization step (update the parameters).
        opt.step()

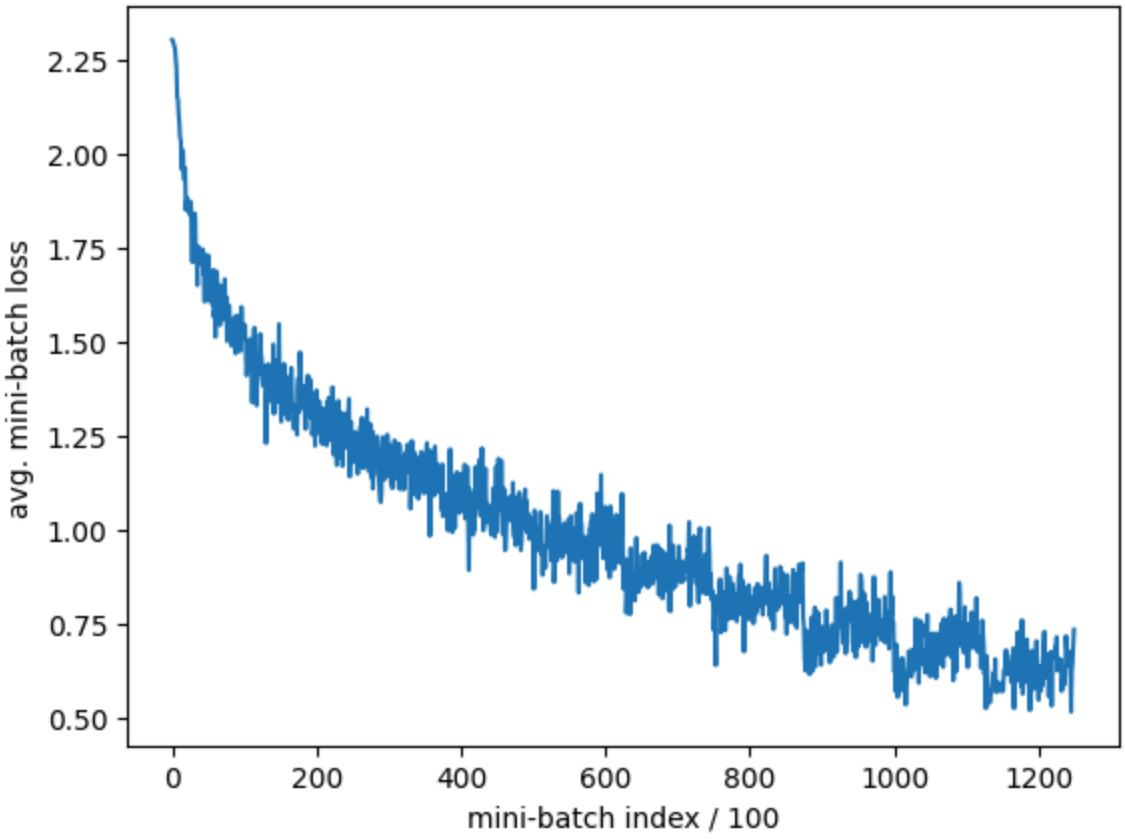
        # Print statistics.
        running_loss += loss.item()
        if i % print_freq == print_freq - 1: # Print every several mini-batches.
            avg_loss = running_loss / print_freq
            print('[epoch: {}, i: {:5d}] avg mini-batch loss: {:.3f}'.format(
                epoch, i, avg_loss))
            avg_losses.append(avg_loss)
            running_loss = 0.0

print('Finished Training.')
```

```
[epoch: 9, i: 10799] avg mini-batch loss: 0.571
[epoch: 9, i: 10899] avg mini-batch loss: 0.603
[epoch: 9, i: 10999] avg mini-batch loss: 0.584
[epoch: 9, i: 11099] avg mini-batch loss: 0.604
[epoch: 9, i: 11199] avg mini-batch loss: 0.590
[epoch: 9, i: 11299] avg mini-batch loss: 0.625
[epoch: 9, i: 11399] avg mini-batch loss: 0.718
[epoch: 9, i: 11499] avg mini-batch loss: 0.686
[epoch: 9, i: 11599] avg mini-batch loss: 0.681
[epoch: 9, i: 11699] avg mini-batch loss: 0.652
[epoch: 9, i: 11799] avg mini-batch loss: 0.676
[epoch: 9, i: 11899] avg mini-batch loss: 0.637
[epoch: 9, i: 11999] avg mini-batch loss: 0.638
[epoch: 9, i: 12099] avg mini-batch loss: 0.516
[epoch: 9, i: 12199] avg mini-batch loss: 0.622
[epoch: 9, i: 12299] avg mini-batch loss: 0.648
[epoch: 9, i: 12399] avg mini-batch loss: 0.696
[epoch: 9, i: 12499] avg mini-batch loss: 0.735
Finished Training.
```

Training Loss Curve

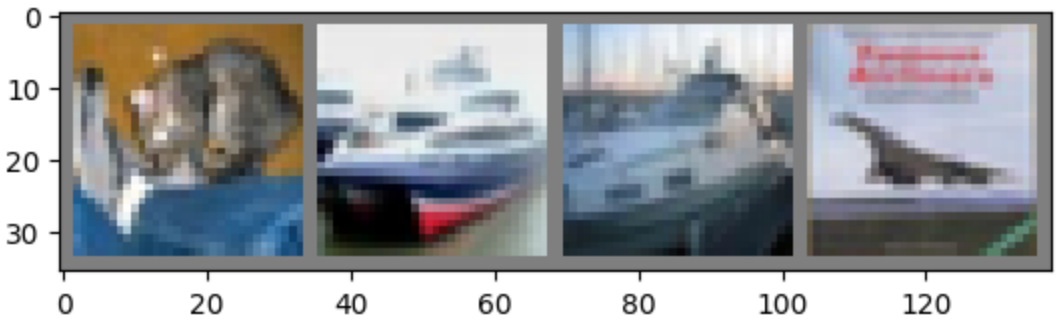
In [8]:
```
plt.plot(avg_losses)
plt.xlabel('mini-batch index / {}'.format(print_freq))
plt.ylabel('avg. mini-batch loss')
plt.show()
```



Evaluate on Test Dataset

In [11]:
```
# Check several images.
dataiter = iter(testloader)
images, labels = next(dataiter)
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(4)))
outputs = net(images.to(device))
_, predicted = torch.max(outputs, 1)

print('Predicted: ', ' '.join('%5s' % classes[predicted[j]]
                              for j in range(4)))
```




```
GroundTruth:    cat  ship  ship plane
Predicted:   ship   car  ship plane
```

In [12]:
```
# Get test accuracy.
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print('Accuracy of the network on the 10000 test images: %d %%' % (
    100 * correct / total))
```

```
Accuracy of the network on the 10000 test images: 65 %
```

In [13]:
```
# Get test accuracy for each class.
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = net(images)
        _, predicted = torch.max(outputs, 1)
        c = (predicted == labels).squeeze()
        for i in range(4):
            label = labels[i]
            class_correct[label] += c[i].item()
            class_total[label] += 1
```

```
for i in range(10):
    print('Accuracy of %5s : %2d %%' % (
        classes[i], 100 * class_correct[i] / class_total[i]))
```

```
Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__ at 0x7da492d9
3c40>
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", lin
e 1654, in __del__
    self._shutdown_workers()
  File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", lin
e 1637, in _shutdown_workers
    if w.is_alive():
       ^^^^^^^^^^^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in is_alive
    assert self._parent_pid == os.getpid(), 'can only test a child process'
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: can only test a child process
Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__ at 0x7da492d9
3c40>Traceback (most recent call last):

  File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", lin
e 1654, in __del__
    self._shutdown_workers()
  File "/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py", lin
e 1637, in _shutdown_workers
    if w.is_alive():
       ^^^^^^^^^^^^
  File "/usr/lib/python3.12/multiprocessing/process.py", line 160, in is_alive
    assert self._parent_pid == os.getpid(), 'can only test a child process'
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: can only test a child process
Accuracy of plane : 75 %
Accuracy of   car : 73 %
Accuracy of  bird : 46 %
Accuracy of   cat : 44 %
Accuracy of  deer : 70 %
Accuracy of   dog : 44 %
Accuracy of  frog : 64 %
Accuracy of horse : 71 %
Accuracy of  ship : 81 %
Accuracy of truck : 79 %
```

Problem 6

In [115…
```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
import torch
import torchvision
import torchvision.transforms as transforms
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

Prepare for Dataset

In [116…
```
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                          shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
                                         shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

Choose a Device

In [117…
```
# If there are GPUs, choose the first one for computing. Otherwise use CPU.
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(device)
# If 'cuda:0' is printed, it means GPU is available.
```

```
cuda:0
```

Network Definition

In [118…
```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        ###### Fill the blank here ######
        self.conv1 = nn.Conv2d(3, 32, 3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, 3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
```

```
        self.conv3 = nn.Conv2d(64, 128, 3, stride=1, padding=1)
        self.bn3 = nn.BatchNorm2d(128)

        self.pool = nn.MaxPool2d(2, 2)
        self.dropout_conv = nn.Dropout2d(0.25)

        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.dropout_fc = nn.Dropout(0.5)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        ###### Fill the blank here ######
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.dropout_conv(x)
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = self.dropout_conv(x)
        x = self.pool(F.relu(self.bn3(self.conv3(x))))
        x = self.dropout_conv(x)

        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout_fc(x)
        x = self.fc2(x)
        return x

net = Net()     # Create the network instance.
net.to(device)  # Move the network parameters to the specified device.
```

Out[118…

```
Net(
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats
=True)
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats
=True)
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (bn3): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stat
s=True)
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=Fals
e)
  (dropout_conv): Dropout2d(p=0.25, inplace=False)
  (fc1): Linear(in_features=2048, out_features=256, bias=True)
  (dropout_fc): Dropout(p=0.5, inplace=False)
  (fc2): Linear(in_features=256, out_features=10, bias=True)
)
```

Optimizer and Loss Function

In [119…

```
loss_func = nn.CrossEntropyLoss()
opt = optim.Adam(net.parameters(), lr=1e-3)
scheduler = optim.lr_scheduler.StepLR(opt, step_size=7, gamma=0.5)
```

Training Procedure

In [120…

```
avg_losses = []   # Avg. losses.
epochs = 25       # Total epochs.
print_freq = 1000  # Print frequency.

for epoch in range(epochs):  # Loop over the dataset multiple times.
    running_loss = 0.0       # Initialize running loss.
    for i, data in enumerate(trainloader, 0):
        # Get the inputs.
        inputs, labels = data

        # Move the inputs to the specified device.
        inputs, labels = inputs.to(device), labels.to(device)

        # Zero the parameter gradients.
        opt.zero_grad()

        # Forward step.
        outputs = net(inputs)
        loss = loss_func(outputs, labels)

        # Backward step.
        loss.backward()

        # Optimization step (update the parameters).
        opt.step()

        # Print statistics.
        running_loss += loss.item()
        if i % print_freq == print_freq - 1: # Print every several mini-batches.
            avg_loss = running_loss / print_freq
            print('[epoch: {}, i: {:5d}] avg mini-batch loss: {:.3f}'.format(
                epoch, i, avg_loss))
            avg_losses.append(avg_loss)
            running_loss = 0.0
    scheduler.step()

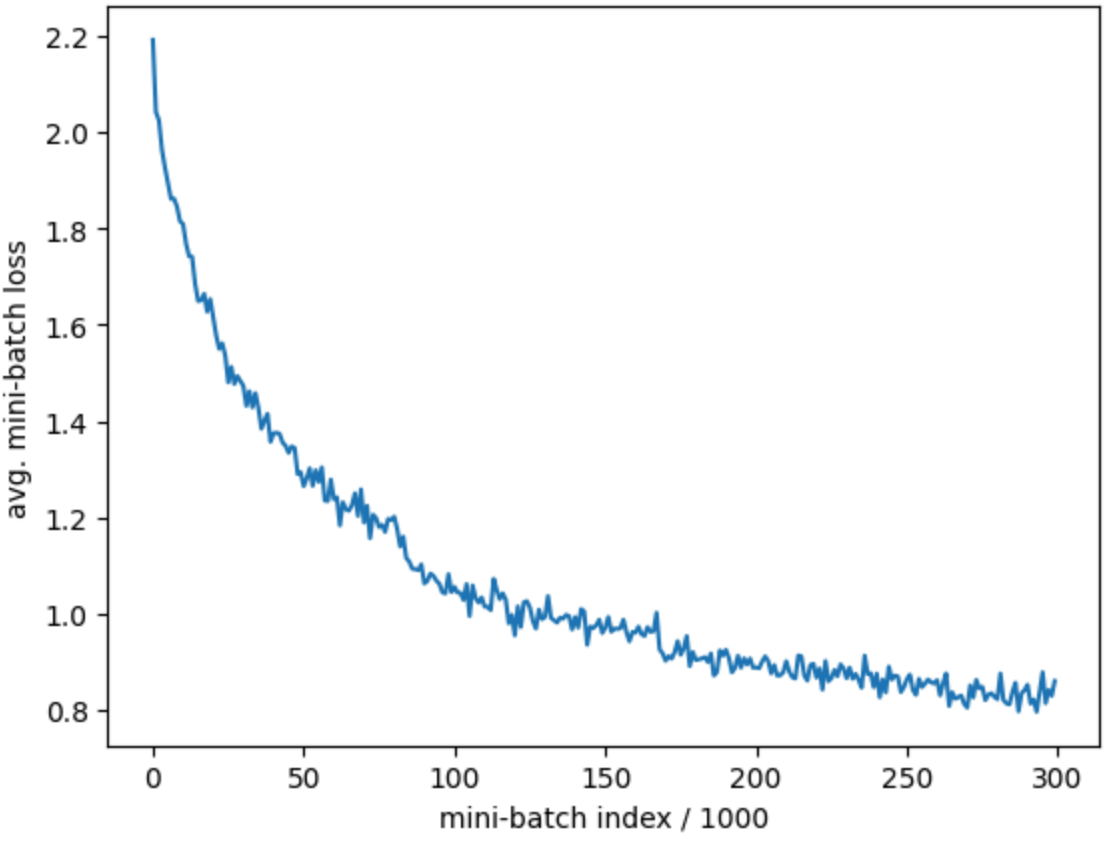
print('Finished Training.')
```

```
[epoch: 23, i:  4999] avg mini-batch loss: 0.823
[epoch: 23, i:  5999] avg mini-batch loss: 0.876
[epoch: 23, i:  6999] avg mini-batch loss: 0.821
[epoch: 23, i:  7999] avg mini-batch loss: 0.814
[epoch: 23, i:  8999] avg mini-batch loss: 0.812
[epoch: 23, i:  9999] avg mini-batch loss: 0.838
[epoch: 23, i: 10999] avg mini-batch loss: 0.857
[epoch: 23, i: 11999] avg mini-batch loss: 0.798
[epoch: 24, i:   999] avg mini-batch loss: 0.835
[epoch: 24, i:  1999] avg mini-batch loss: 0.842
[epoch: 24, i:  2999] avg mini-batch loss: 0.852
[epoch: 24, i:  3999] avg mini-batch loss: 0.814
[epoch: 24, i:  4999] avg mini-batch loss: 0.822
[epoch: 24, i:  5999] avg mini-batch loss: 0.796
[epoch: 24, i:  6999] avg mini-batch loss: 0.838
[epoch: 24, i:  7999] avg mini-batch loss: 0.879
[epoch: 24, i:  8999] avg mini-batch loss: 0.815
[epoch: 24, i:  9999] avg mini-batch loss: 0.842
[epoch: 24, i: 10999] avg mini-batch loss: 0.830
[epoch: 24, i: 11999] avg mini-batch loss: 0.860
Finished Training.
```

Training Loss Curve

In [121…

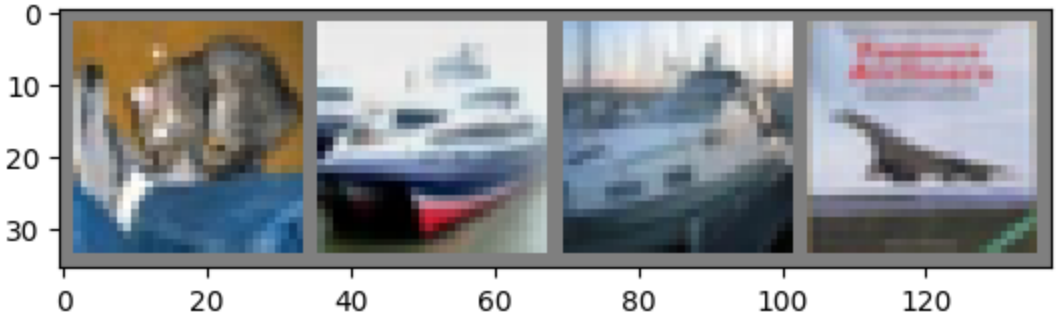
```
plt.plot(avg_losses)
plt.xlabel('mini-batch index / {}'.format(print_freq))
plt.ylabel('avg. mini-batch loss')
plt.show()
```



Evaluate on Test Dataset

In [122…
```
# Check several images.
dataiter = iter(testloader)
images, labels = next(dataiter)
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(4)))
outputs = net(images.to(device))
_, predicted = torch.max(outputs, 1)

print('Predicted: ', ' '.join('%5s' % classes[predicted[j]]
                              for j in range(4)))
```




```
GroundTruth:    cat  ship  ship plane
Predicted:    dog  ship  ship plane
```

In [123…
```
# Get test accuracy.
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print('Accuracy of the network on the 10000 test images: %d %%' % (
    100 * correct / total))
```

```
Accuracy of the network on the 10000 test images: 67 %
```

In [124…
```
# Get test accuracy for each class.
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = net(images)
        _, predicted = torch.max(outputs, 1)
        c = (predicted == labels).squeeze()
        for i in range(4):
            label = labels[i]
```

```
            class_correct[label] += c[i].item()
            class_total[label] += 1

for i in range(10):
    print('Accuracy of %5s : %2d %%' % (
        classes[i], 100 * class_correct[i] / class_total[i]))
```

```
Accuracy of plane : 68 %
Accuracy of   car : 78 %
Accuracy of  bird : 51 %
Accuracy of   cat : 49 %
Accuracy of  deer : 64 %
Accuracy of   dog : 54 %
Accuracy of  frog : 73 %
Accuracy of horse : 66 %
Accuracy of  ship : 78 %
Accuracy of truck : 75 %
```